핀홀 카메라 모델

Camera Calibration

cchyun@gmail.com

Part 1. 핀홀 카메라 모델

3D 세계를 2D 이미지로 바꾸는 원리

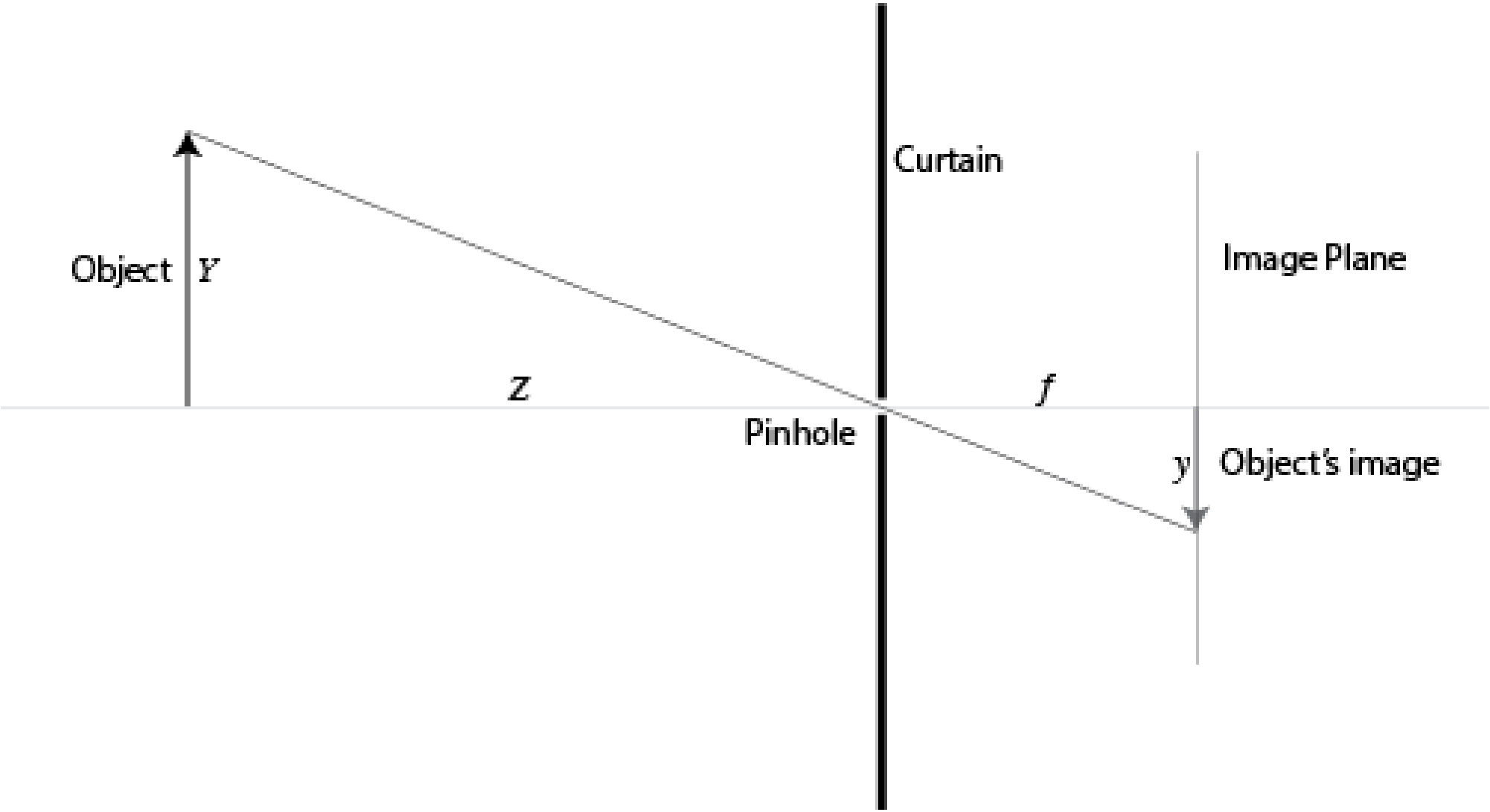
- **"빛은 직선으로 나아간다"는 단순한 원리로 3D → 2D 변환**

  - **바늘구멍 사진기 (Pinhole Camera)**
    - 아주 캄캄한 상자 앞면에 바늘구멍(Pinhole) 하나를 뚫음
    - 물체에서 반사된 빛이 구멍을 통과하여 상자 뒷면(센서)에 맺힘

  - **투영 (Projection)**
    - 3D 공간의 한 점이 구멍을 통과해 2D 평면의 한 점으로 **내려앉는** 과정

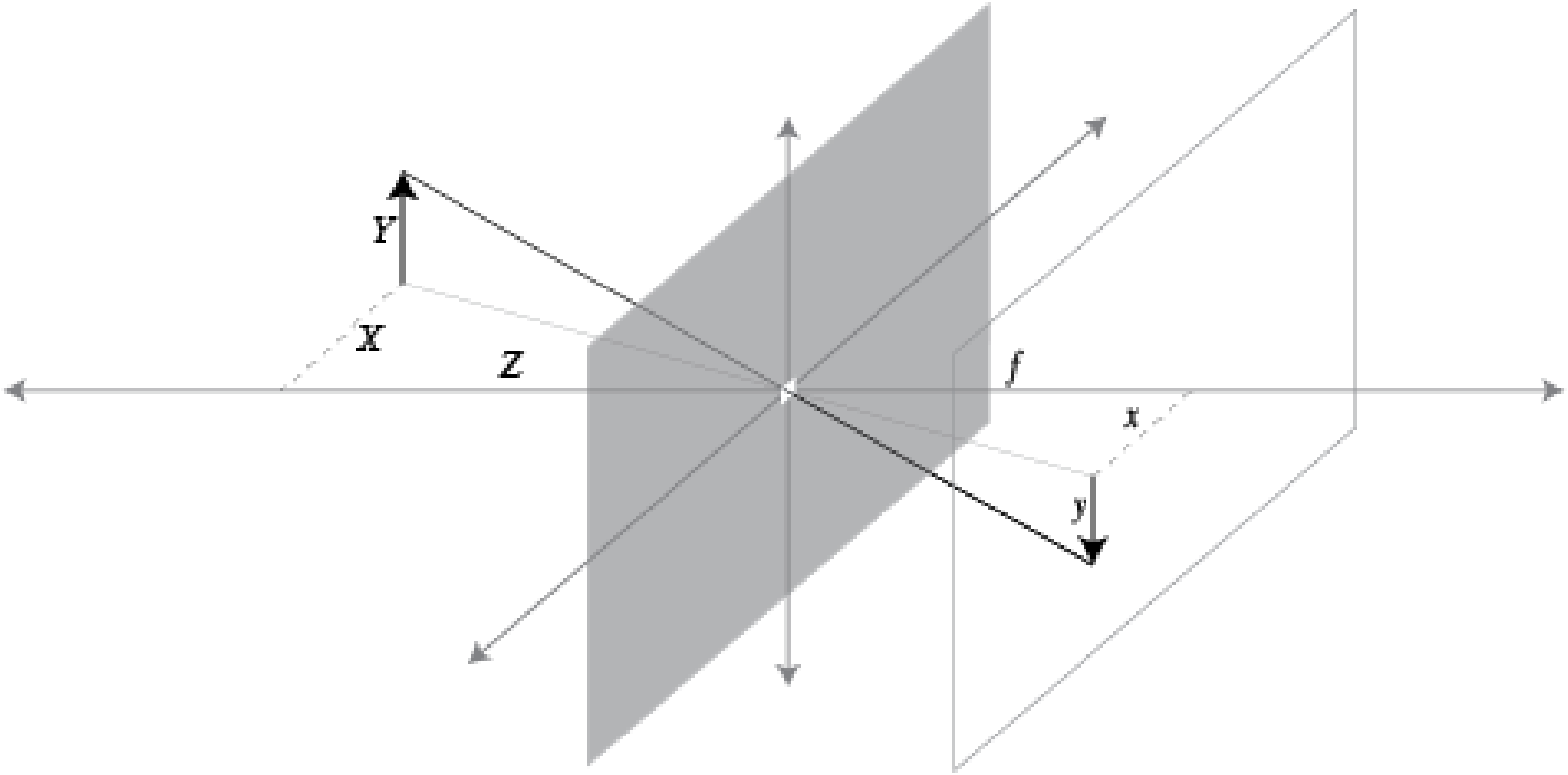
  - **로봇의 거리 추정**
    - 로봇은 이 원리를 **거꾸로** 이용
    - 2D 이미지의 한 점 → 실제 3D 공간의 어떤 선상에서 온 것인지 분석
    - 2D로 변환되는 순간 **'깊이' 정보가 사라짐** → 수학적 모델로 재계산 필요

핀홀 카메라 모델 (2D)



https://csundergrad.science.uoit.ca/courses/cv-notes/notebooks/01-image-formation.html

핀홀 카메라 모델 (3D)



https://csundergrad.science.uoit.ca/courses/cv-notes/notebooks/01-image-formation.html

Aperture (조리개) — 핀홀의 구멍 크기

- **Aperture : 카메라 렌즈에서 빛이 통과하는 구멍의 크기**
  - **크기에 따른 trade-off**

Aperture 크기	빛의 양	이미지 선명도
크다 (2mm)	많음	흐릿
작다 (0.35mm)	적음	선명

- **핀홀 카메라 모델의 가정**
  - Aperture = **수학적으로 크기가 0인 점**
  - 모든 빛이 딱 한 점을 통과 → 깔끔한 투영 수식 성립
  - 현실에서는 너무 작으면 **빛 부족** → **어두운 이미지** 문제 발생

렌즈 (Lens) — Aperture 딜레마의 해결책

- **렌즈가 핀홀의 문제를 해결하는 원리**

  - **핀홀의 딜레마**

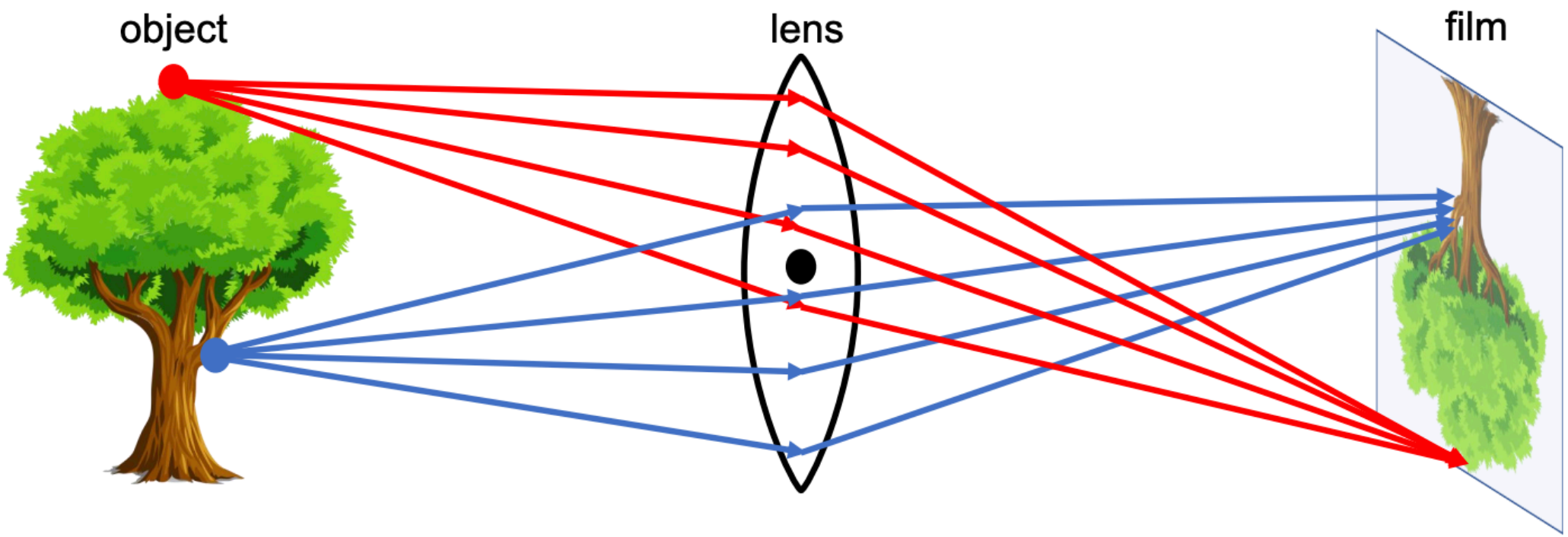
```
구멍 크게 → 빛 많음  but  흐릿
구멍 작게 → 선명     but  어두움
```

  - **렌즈의 해결책**
    - 한 점에서 나온 **여러 줄기 빛**을 모아 film의 **한 점으로 수렴**

```
빨간 점 → 여러 빛줄기 → 렌즈 → film의 한 점
파란 점 → 여러 빛줄기 → 렌즈 → film의 다른 점
```

  - **새로운 문제**
    - 렌즈의 굴절 특성 때문에 → **왜곡(Distortion)** 발생
    - 이를 수학적으로 보정하는 것이 **캘리브레이션**

렌즈 (Lens) — Aperture 딜레마의 해결책



https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf

렌즈와 초점 거리 — 물체 거리와 Focal Point

- **물체 거리에 따라 focal point 위치가 달라지는 이유**
  - **빛의 입사 각도**가 핵심

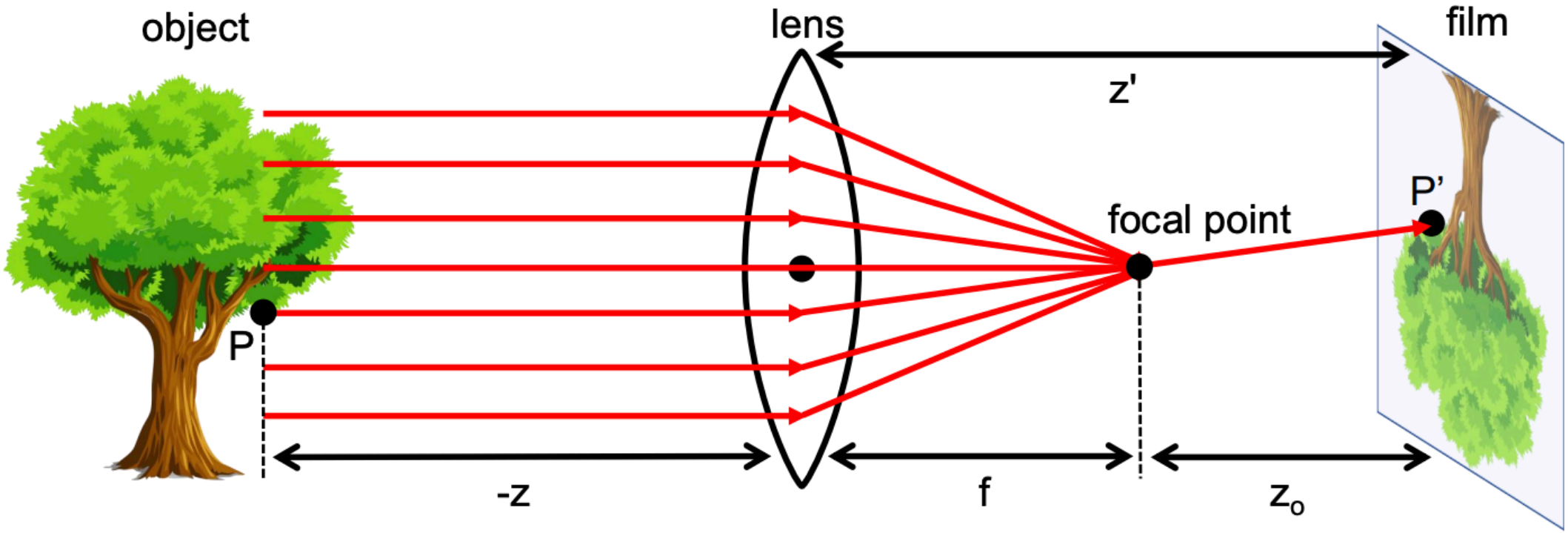
물체 거리	렌즈 입사 각도	Focal Point 위치
멀리	거의 평행	렌즈에 가까움
가까이	퍼져서 입사	렌즈에서 멀어짐

- **렌즈 공식**

$$\frac{1}{f} = \frac{1}{z'} - \frac{1}{z}$$

- **일상에서의 예**
  - 스마트폰 탭해서 초점 맞추기 → 렌즈 위치 조정
  - DSLR 포커스 링 돌리기 → 렌즈를 앞뒤로 이동

렌즈와 초점 거리 — 물체 거리와 Focal Point



https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf

핵심 개념 — 초점 거리 (Focal Length)

- **초점 거리 f : 카메라의 '확대율'을 결정하는 핵심 값**
  - **의미**
    - 카메라의 광학 중심(구멍)과 이미지 평면(센서) 사이의 거리
  - **역할**
    - 초점 거리가 **길면** → 물체가 크게 보임 (망원)
    - 초점 거리가 **짧으면** → 넓은 범위를 담음 (광각)
  - **로봇 비전에서의 의미**
    - 이미지 상의 픽셀 크기를 **실제 물리적 거리와 연결**하는 중요한 비율값
    - 카메라 행렬 K에서 **픽셀 단위** (f_x, f_y)로 표현됨 (mm 단위 아님!)

핵심 개념 — f에서 f_x, f_y로

- **왜 하나의 f가 f_x, f_y 두 개가 되는가?**
  - **픽셀이 정사각형이 아닐 수 있기 때문**
    - d_x : 픽셀 하나의 **가로** 실제 크기 (mm/pixel)
    - d_y : 픽셀 하나의 **세로** 실제 크기 (mm/pixel)
    - $d_x \neq d_y$ 이면 → $f_x \neq f_y$

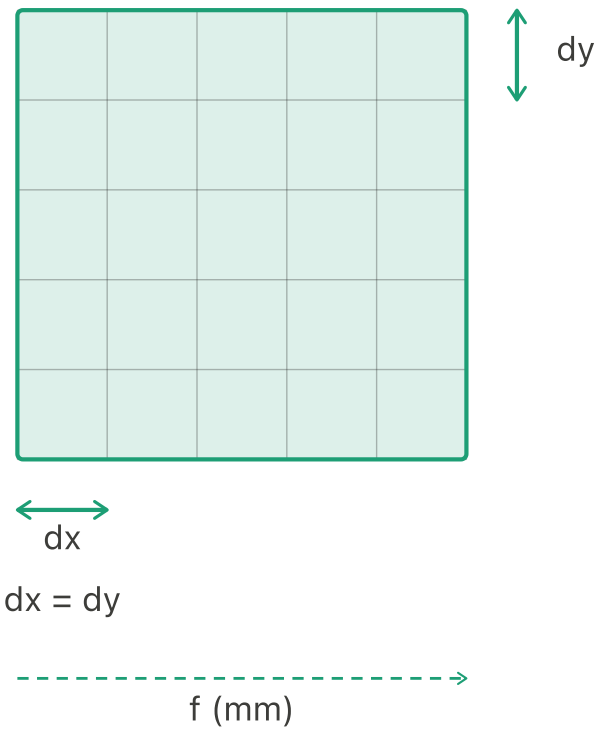
$$f_x = \frac{f}{d_x}, \quad f_y = \frac{f}{d_y}$$

- **의미 정리**

기호	단위	의미
f	mm	O → C' 물리적 거리
d_x, d_y	mm/pixel	픽셀 하나의 실제 크기
f_x, f_y	pixel	투영 수식의 스케일 인자

핵심 개념 — f에서 f_x, f_y로

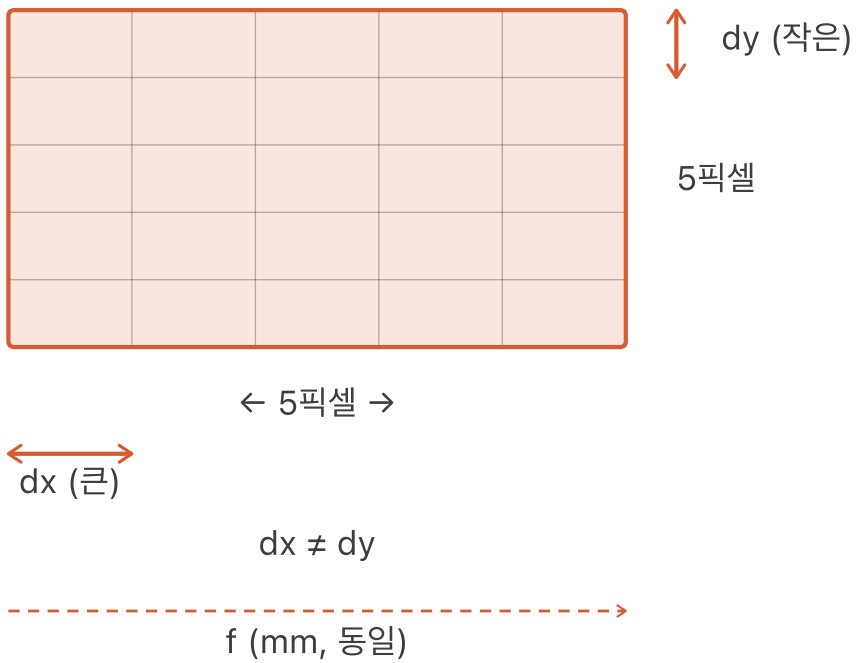
정사각형 픽셀 (dx = dy)



fx = f / dx
fy = f / dy

∴ fx = fy ✓

직사각형 픽셀 (dx ≠ dy)



fx = f / dx (작음)
fy = f / dy (큼)

∴ fx ≠ fy

핵심 개념 — 주점 (Principal Point)

- **주점 (c_x, c_y) : 카메라 렌즈 중심이 이미지와 만나는 기준점**
  - **의미**
    - 카메라 렌즈의 중심(광축)이 이미지 센서와 수직으로 만나는 지점의 픽셀 좌표
  - **이상과 현실**
    - 이상적인 카메라라면 **이미지의 정중앙**이어야 함
    - 실제 제조 과정에서의 오차로 **조금씩 치우쳐** 있음
  - **로봇 비전에서의 의미**
    - (0, 0) 좌표가 아닌 **실제 렌즈 중심으로부터의 거리**를 계산하기 위한 기준점
    - 이 값이 틀리면 → 로봇이 물체 위치를 잘못 파악!

(0, 0)은 왼쪽 상단 모서리

핵심 개념 — 내부 파라미터 행렬 (Camera Matrix K)

- **카메라 행렬 K : 카메라의 고유한 '명함'**
  - **구성**
    - 초점 거리 (f_x, f_y) 와 주점 (c_x, c_y) 정보를 하나로 묶은 **3×3 행렬**

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **의미**
  - "이 카메라의 렌즈와 센서 특성상, 3D 좌표는 이런 비율로 2D 픽셀로 변한다"는 약속
  - 카메라마다 고유한 값을 가지며, **캘리브레이션으로 측정**

3D → 2D 변환 예시

- **설정값:**

$$K = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}, \quad P_{3D} = (X, Y, Z) = (0.3, 0.2, 2.0)\ \mathrm{m}$$

- **수식:**

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}$$

- **계산:**

$$u = f_x \cdot \frac{X}{Z} + c_x = 800 \times \frac{0.3}{2.0} + 320 = 120 + 320 = \mathbf{440}$$

$$v = f_y \cdot \frac{Y}{Z} + c_y = 800 \times \frac{0.2}{2.0} + 240 = 80 + 240 = \mathbf{320}$$

- 3D 점 **(0.3, 0.2, 2.0m)** 는 이미지의 픽셀 **(440, 320)** 에 찍힘

2D → 3D 역변환 예시 (Z를 아는 경우)

- **설정값:**
  - 픽셀 좌표 $(u, v) = (440, 320)$, $Z = 2.0$ m (depth sensor 등으로 측정)

- **수식 (K의 역연산):**

$$X = \frac{(u - c_x)}{f_x} \cdot Z = \frac{(440 - 320)}{800} \times 2.0 = \frac{120}{800} \times 2.0 = \mathbf{0.3}\ \mathrm{m}$$

$$Y = \frac{(v - c_y)}{f_y} \cdot Z = \frac{(320 - 240)}{800} \times 2.0 = \frac{80}{800} \times 2.0 = \mathbf{0.2}\ \mathrm{m}$$

- 픽셀 **(440, 320)** + 깊이 **2.0m** → 3D 점 **(0.3, 0.2, 2.0m)** 복원 성공!

실제 카메라 vs 수학 모델

- **핀홀 수식을 쓰지만 실제로는 렌즈 카메라를 사용**
  - **현실** : 렌즈 카메라 (밝고 선명)
  - **수학 모델** : 핀홀 모델 (수식이 단순하고 깔끔)
  - **캘리브레이션이 다리 역할**

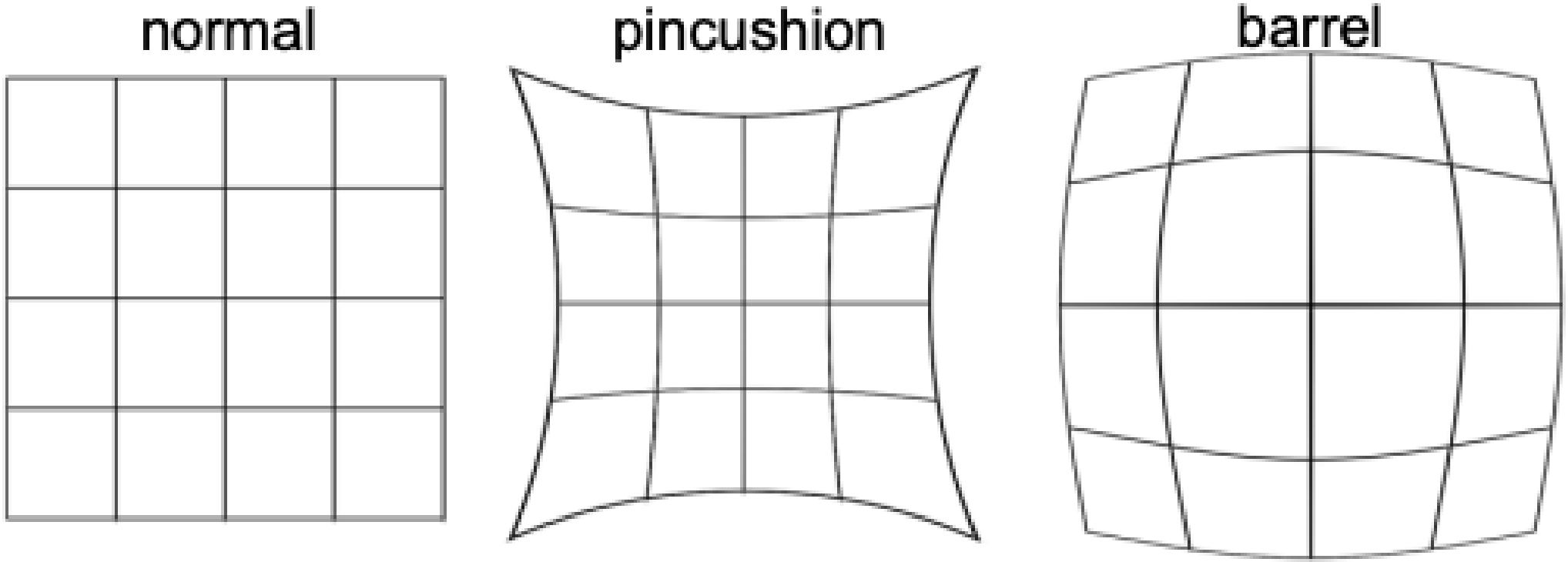
```
실제 렌즈 카메라
    ↓
캘리브레이션으로 왜곡 계수(k1, k2, p1, p2, k3?) 측정
    ↓
왜곡 보정 (Rectification)
    ↓
핀홀 모델 수식 적용
```

- **핵심**
  - 왜곡을 제거하고 나면 → 렌즈 카메라도 **핀홀 카메라처럼 동작**

렌즈 왜곡의 종류 — 방사 왜곡 (Radial Distortion)

- **방사 왜곡 (Radial Distortion)**
  - **원인**
    - 렌즈의 **곡률(curvature) 특성** 때문에 가장자리로 갈수록 직선이 곡선으로 변하는 현상
  - **배럴 왜곡 (Barrel Distortion)**
    - 이미지가 **볼록하게 튀어나와** 보이는 현상
    - 주로 **광각 렌즈**에서 나타남
  - **핀쿠션 왜곡 (Pincushion Distortion)**
    - 이미지가 **오목하게 들어가** 보이는 현상
    - 주로 **망원 렌즈**에서 나타남
  - **보정 계수**
    - 방사 왜곡은 **k1, k2, k3** 로 보정
    - 렌즈 중심에서 멀어질수록 왜곡이 커지므로, **거리의 제곱** 단위로 보정값 적용

렌즈 왜곡의 종류 — 방사 왜곡 (Radial Distortion)



https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf

렌즈 왜곡의 종류 — 접선 왜곡 (Tangential Distortion)

- **접선 왜곡 (Tangential Distortion)**

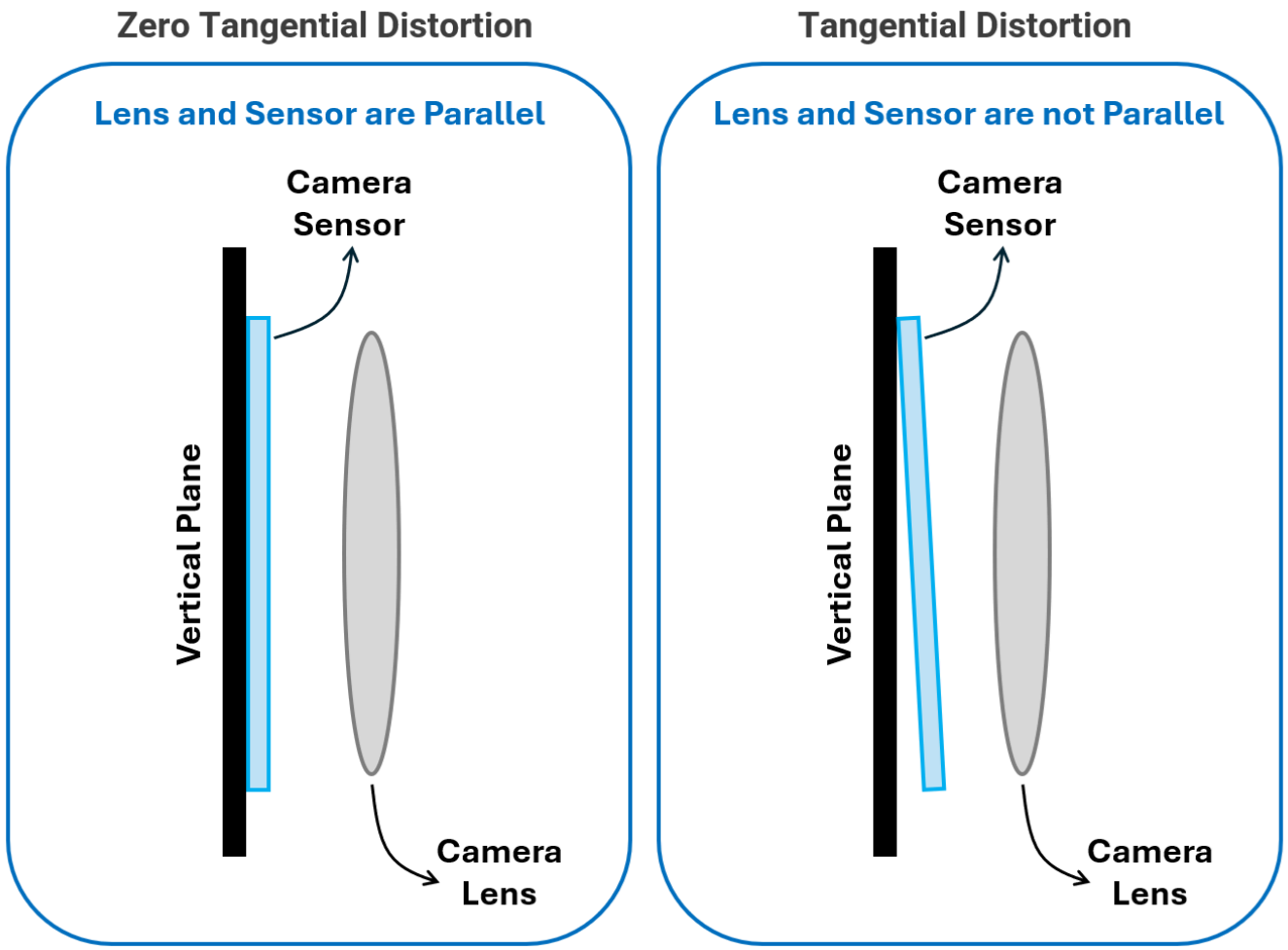
  - **원인**
    - 카메라 제조 시 렌즈와 이미지 센서가 **완벽하게 수평을 이루지 못해** 이미지가 비스듬하게 기울어져 보이는 현상

  - **로봇 비전에 미치는 영향**
    - 왜곡된 이미지의 픽셀 좌표를 그대로 쓰면 **거리 측정 오차**가 크게 발생
    - 예: 실제로 일직선인 복도가 휘어 보임 → 로봇이 경로를 잘못 설정

  - **해결책**
    - **카메라 캘리브레이션(Calibration)** 을 통해 왜곡을 수학적으로 모델링
    - **직선화(Rectification)** 작업 필수

  - **보정 계수 : p1, p2**
    - **p1** : 세로 방향(y축) 기울기로 인한 왜곡 보정
    - **p2** : 가로 방향(x축) 기울기로 인한 왜곡 보정

렌즈 왜곡의 종류 — 접선 왜곡 (Tangential Distortion)



https://blog.roboflow.com/vision-ai-camera-calibration/

좌표계의 종류와 변환 흐름

- **픽셀 하나를 보고 실제 위치를 알기까지 4단계 좌표 변환**

  - **① 3D 월드 좌표계 (World Frame)**
    - 로봇이 움직이는 전체 공간의 기준 (예: 방의 구석이 0, 0, 0)

  - **② 3D 카메라 좌표계 (Camera Frame)**
    - 카메라 렌즈의 중심을 기준으로 한 3D 좌표
    - 월드 → 카메라 변환 : **외부 파라미터(Extrinsics)**

  - **③ 2D 이미지 좌표계 (Image Plane)**
    - 3D 공간의 점이 렌즈를 통해 투영된 가상의 2D 평면

  - **④ 픽셀 좌표계 (Pixel Frame)**
    - 파이썬 코드에서 실제로 다루는 (u, v) 좌표
    - 이미지 평면의 물리적 단위(mm) → 카메라 행렬 K로 픽셀 단위로 변환

렌즈 왜곡 보정 수식

- **왜곡된 좌표 → 보정된 좌표 변환** (1/2)

  - **방사 왜곡 보정 (k1, k2, k3)**

$$x' = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)$$
$$y' = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)$$

  - **접선 왜곡 보정 (p1, p2)**

$$x' = x + \left[2p_1 xy + p_2(r^2 + 2x^2)\right]$$
$$y' = y + \left[p_1(r^2 + 2y^2) + 2p_2 xy\right]$$

렌즈 왜곡 보정 수식

- **왜곡된 좌표 → 보정된 좌표 변환** (2/2)

  - **통합 보정 (전체)**

$$x' = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + 2p_1 xy + p_2\left(r^2 + 2x^2\right)$$
$$y' = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + p_1\left(r^2 + 2y^2\right) + 2p_2 xy$$

  - **여기서**
    - (x, y) : **정규화 좌표** $- x = X/Z,\ y = Y/Z$ 로 단위가 사라진 비율값
    - $r^2 = x^2 + y^2$: 광축 중심으로부터의 거리 제곱
    - (x', y') : 보정된 정규화 좌표

좌표 변환 흐름 요약

- **"화면 속 픽셀 → 로봇 기준 실제 위치" 과정** (1/2)

  - **1단계 : 픽셀 좌표 → 이미지 좌표**
    - 픽셀에서 카메라 고유 특성 (f_x, f_y, c_x, c_y) 을 제거
    - "이 픽셀이 렌즈 중심 기준으로 **어느 방향**인가?"

$$x = \frac{u - c_x}{f_x}, \quad y = \frac{v - c_y}{f_y}$$

  - **2단계 : 이미지 좌표 → 카메라 좌표**
    - 방향 정보에 **깊이(Z)** 를 곱해 실제 3D 위치 복원
    - 깊이는 RGB-D 카메라 또는 LiDAR로 측정

$$X = x \cdot Z, \quad Y = y \cdot Z$$

좌표 변환 흐름 요약

- **"화면 속 픽셀 → 로봇 기준 실제 위치" 과정** (2/2)

  - **3단계 : 카메라 좌표 → 월드 좌표**
    - 카메라 기준 위치를 **세계 기준 위치**로 변환
    - 로봇이 어디에(t) 어느 방향으로(R) 있는지 반영

$$P_{world} = R^{-1}(P_{camera} - t)$$

  - **전체 흐름**

$$\text{픽셀 좌표} \xrightarrow{K^{-1}} \text{이미지 좌표} \xrightarrow{\times Z} \text{카메라 좌표} \xrightarrow{[R|t]^{-1}} \text{월드 좌표}$$

  - **결론**
    - 이 3단계를 거쳐야 "화면 속 사과 픽셀이
      월드 좌표 기준 (1.2, 0.0, 1.5)m 위치에 있다"는 결론에 도달

입문자가 가장 많이 혼란스러워하는 포인트

- **혼란 1: 이미지가 거꾸로 맺히는 현상**
  - 물리적으로는 센서에 상이 **거꾸로** 맺히지만
  - 수학적 모델에서는 계산 편의를 위해 이미지 평면을 **렌즈 앞**에 놓고 똑바로 선 이미지를 사용

- **혼란 2: 픽셀(Pixel)과 미터(Meter)의 혼동**
  - 카메라 행렬 K의 초점 거리는 **픽셀 단위** (f_x, f_y)로 환산된 값 (mm 아님!)
  - 실제 거리를 구하려면 픽셀 단위 → 미터 단위로 바꾸는 비례식 필요

- **혼란 3: Optical Frame의 축 방향**
  - 로봇 본체(camera_link): 앞쪽 = **X축**, 위쪽 = **Z축**
  - 카메라 시선 기준(camera_optical_frame): 앞쪽(시선) = **Z축**, 오른쪽 = **X축**, 아래쪽 = **Y축**
  - 좌표가 90도 돌아가 있는 이유 = 이 **축 정의의 차이**

Part 2. 카메라 캘리브레이션 실전

캘리브레이션이 필요한 이유

- **보정되지 않은 카메라는 직선을 곡선으로 인식하거나 물체 위치를 잘못 파악**

  - **캘리브레이션 전**
    - 방사 왜곡(Radial Distortion): 이미지 가장자리에서 직선이 휘어짐
    - 접선 왜곡(Tangential Distortion): 렌즈·센서 비정렬로 이미지가 기울어짐
    - 2D 픽셀 → 3D 좌표로 **정확한 변환 불가**

  - **캘리브레이션 후**
    - 왜곡 계수로 이미지 **직선화(Rectification)** 가능
    - 픽셀 좌표 ↔️ 실제 3D 공간 사이의 **신뢰할 수 있는 수학적 관계** 정의
    - 거리 측정 및 3D 복원 가능

체커보드 패턴 사용 이유

- **체커보드는 알고리즘이 교차점을 서브픽셀 정밀도로 감지할 수 있는 기준점**
  - **왜 체커보드인가?**
    - 명암 대비가 명확 → **내부 교차점(Corners)** 을 하위 픽셀 단위로 정밀 감지 가능
    - 규칙적인 격자 구조 → 실제 3D 좌표를 쉽게 정의 가능
  - **주의: 칸 수가 아닌 내부 교차점의 개수!**
    - 10×7 칸 체커보드 → 내부 교차점은 9×6개
    - 코드에 잘못된 값을 입력하면 코너 검출 실패!

https://raw.githubusercontent.com/opencv/opencv/refs/heads/4.x/doc/pattern.png

올바른 체커보드 촬영 방법

- **캘리브레이션 품질은 촬영 방법에 달려있음**
  - **장수**: 최소 **10~20장** 권장, 복잡한 왜곡 렌즈는 최대 40장
  - **각도**: 체커보드를 **다양한 축 방향**으로 기울여 촬영
    - ±15°~45° 범위 권장
    - 다양한 각도가 있어야 초점 거리를 정확히 계산 가능
  - **거리**: 카메라와 체커보드 사이의 **거리를 변화**시키며 촬영
  - **주의사항**
    - 체커보드는 반드시 **평평하고 단단한 판**에 부착 (구부러지면 안 됨)
    - 촬영 내내 **줌(Zoom) 수준 일정 유지** (줌이 바뀌면 파라미터 무효)
    - 이미지 **전체 영역** (특히 왜곡이 심한 가장자리와 모서리)을 골고루 커버

OpenCV 캘리브레이션 코드 흐름

- **OpenCV 캘리브레이션 3단계**
  - ① `findChessboardCorners()`
    - 이미지에서 체커보드 내부 교차점들의 **픽셀 좌표** 탐색
  - ② `cornerSubPix()`
    - 감지된 코너 좌표를 **하위 픽셀 단위로 정밀화** → 정확도 향상
  - ③ `calibrateCamera()`
    - 2D 이미지 포인트 + 미리 정의된 3D 객체 포인트를 비교
    - 시스템 방정식을 풀어 **카메라 파라미터 계산**

calibrateCamera() 결과 파라미터

```
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None
)
```

- **반환값 설명**
  - `ret` : 재투영 오차 (RMS Error) — 작을수록 좋음
  - `mtx` : 카메라 행렬 (Intrinsic Matrix K, 3×3)
    - 초점 거리 (f_x, f_y) + 주점 (c_x, c_y) 포함
  - `dist` : 왜곡 계수 (Distortion Coefficients)
    - 방사 왜곡 (k_1, k_2, k_3) + 접선 왜곡 (p_1, p_2)
  - `rvecs`, `tvecs` : 각 이미지의 외부 파라미터 (회전, 이동 벡터)

undistort()로 왜곡 보정하기

```
# 왜곡 보정된 이미지 생성
dst = cv2.undistort(img, mtx, dist, None, mtx)
```

- **동작 원리**
  - 계산된 카메라 행렬(K)과 왜곡 계수(`dist`)를 입력
  - **렌즈 왜곡이 제거된 보정된 이미지** 출력
  - 곡선으로 보이던 선들이 다시 **직선으로 복원**
- **결과**
  - 로봇은 보정된 이미지를 바탕으로 **정확한 거리와 방향** 계산 가능

결과 저장 및 불러오기

- 캘리브레이션 결과는 매번 계산할 필요 없이 파일로 저장하여 재사용

- 저장 방법

```
# YAML 형식으로 저장 (ROS 호환)
import yaml
data = {'camera_matrix': mtx.tolist(), 'dist_coeff': dist.tolist()}
with open('camera_info.yaml', 'w') as f:
    yaml.dump(data, f)
```

- 불러오기

```
# YAML 로드
with open('camera_info.yaml', 'r') as f:
    data = yaml.safe_load(f)
mtx  = np.array(data['camera_matrix'], np.float64)
dist = np.array(data['dist_coeff'], np.float64)
# 실시간 영상에서 undistort 적용
dst = cv2.undistort(frame, mtx, dist, None, mtx)
```

품질 평가 — 재투영 오차 (Reprojection Error)

- **재투영 오차(Reprojection Error) : 캘리브레이션 정확도의 핵심 지표**

  - **정의**
    - 계산된 카메라 모델로 3D 포인트를 다시 2D에 투영했을 때
    - **실제 감지된 이미지 포인트와의 거리 차이(픽셀 단위)**

  - **평가 기준**
    - **0.5 픽셀 미만** : 산업용 비전 시스템에서 성공적으로 간주
    - **1 픽셀 이하** : 대부분의 응용 분야에서 수용 가능
    - **1 픽셀 초과** : 촬영 데이터를 점검하고 재캘리브레이션 필요

입문자가 자주 하는 실수와 해결법

- **실수 1: 줌/초점 변경**
  - 촬영 도중 줌을 조작하면 내부 파라미터가 변함
  - **해결법**: 촬영 전 줌과 초점을 **고정**한 뒤 손대지 않음

- **실수 2: 가장자리 데이터 부족**
  - 체커보드를 화면 중앙에서만 촬영 → 가장자리 왜곡 계산 부정확
  - **해결법**: 보드가 화면의 **모든 모서리에 걸치도록** 촬영

- **실수 3: 패턴 수 세기 오류**
  - 체커보드의 **칸 수**가 아닌 **내부 교차점 개수**를 정확히 입력해야 함
  - 8×6 칸 → 교차점은 7×5개

- **실수 4: 조명 불량**
  - 어두운 곳 = 노이즈 증가 → 코너 검출 부정확 / 반사광 = 명암 차이 소멸
  - **해결법**: **밝고 균일한 조명** 아래서 그림자·반사 없이 촬영

실제 로봇 카메라 캘리브레이션 실습 순서 (1/2)

- **Step 1. 준비**
  - 체커보드 패턴을 고해상도로 출력하여 **평평하고 단단한 판**에 부착
- **Step 2. 설정**
  - 로봇 카메라의 **해상도, 줌, 초점 고정**
- **Step 3. 데이터 수집**
  - 다양한 각도와 거리에서 **20장 이상** 촬영
  - 보드가 이미지의 **상하좌우 모든 영역을 커버**하도록
- **Step 4. 검출**
  - `findChessboardCorners()` + `cornerSubPix()` 로 **정밀한 코너 좌표** 추출
- **Step 5. 계산**
  - `calibrateCamera()` 실행 → **카메라 행렬과 왜곡 계수** 산출

실제 로봇 카메라 캘리브레이션 실습 순서 (2/2)

- **Step 6. 검증**
  - 재투영 오차(RMS Error) **0.5px 내외** 확인 + 직선 보정 시각화
- **Step 7. 배포**
  - 파라미터를 **YAML 파일로 저장** → 로봇 비전 노드에서 실시간 `undistort()` 적용

Part 3. 체크리스트

실습 전 체크리스트

실습을 시작하기 전, 다음 사항을 모두 확인하세요!

- [ ] **평면성 확인**: 체커보드 패턴을 아크릴판·폼보드 등 **딱딱하고 평평한 판**에 고정했나요?
- [ ] **조명 상태**: 패턴 위에 직접적인 그림자가 지거나, 강한 **반사광**으로 코너가 흐릿해 보이지 않나요?
- [ ] **카메라 설정 고정**: 촬영 도중 **줌을 조절하거나 해상도를 변경**하지 않았나요?
- [ ] **다양한 구도**: 패턴을 이미지의 **상하좌우 가장자리와 모서리 영역까지 골고루** 배치하여 촬영했나요?
- [ ] **기울기 확보**: 패턴을 카메라에 대해 **약 ±45도 정도 기울인** 사진들도 포함시켰나요?
  (초점 거리 계산에 필수!)